

Bayesian and Decision Modelling Coursework

```
In [ ]: import pyagrum as gum
import pyagrum.lib.notebook as gnb
import pyagrum.causal as cs1
import pyagrum.causal.notebook as cs1nb
```

Question 2 - Creating The Bayesian Network

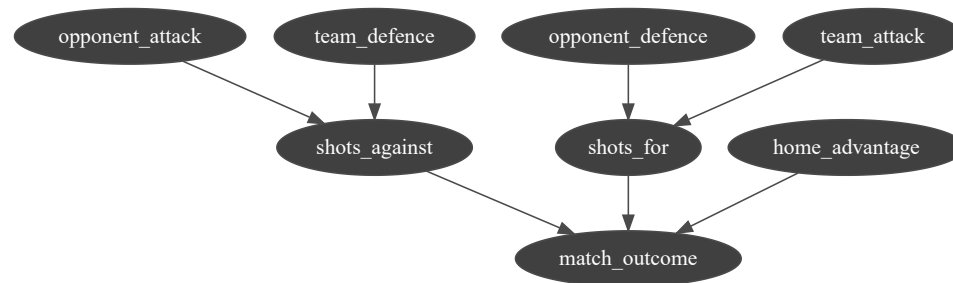
```
In [2]: #Task 2 of Bayesian Coursework - Creating the Bayesian Network

#Create the Bayesian Network
bn = gum.BayesNet("Football_Match_Result")

#Declare all of the nodes
bn.add(gum.LabelizedVariable("match_outcome" , "Match Outcome" , 3))
bn.add(gum.LabelizedVariable("shots_for" , "Shots on Target (For)" , 3))
bn.add(gum.LabelizedVariable("shots_against" , "Shots on Target (Against)" , 3))
bn.add(gum.LabelizedVariable("home_advantage" , "Home Advantage" , 2))
bn.add(gum.LabelizedVariable("team_attack" , "Team Attack Strength" , 3))
bn.add(gum.LabelizedVariable("team_defence" , "Team Defence Strength" , 3))
bn.add(gum.LabelizedVariable("opponent_attack" , "Opponent Attack Strength" , 3))
bn.add(gum.LabelizedVariable("opponent_defence" , "Opponent Defence Strength" , 3))

#Declare all of the arcs/edges between the nodes
bn.addArc("team_attack" , "shots_for")
bn.addArc("opponent_defence" , "shots_for")
bn.addArc("team_defence" , "shots_against")
bn.addArc("opponent_attack" , "shots_against")
bn.addArc("shots_for" , "match_outcome")
bn.addArc("home_advantage" , "match_outcome")
bn.addArc("shots_against" , "match_outcome")
```

```
#Show Bayesian Network Structure
gnb.showBN(bn)
```



Question 3 - Parameterisation of CPTs

In [3]: *#Task 3 of Bayesian Coursework - Defining the CPT's of all nodes*

```
# CPT for root node - home_advantage
```

```
#For Home Advantage:
```

```
# 0 represents 'Away'
```

```
# 1 represents 'Home'
```

```
bn.cpt("home_advantage")[:,] = [0.5, 0.5]
```

```
# CPT for a node with at least 2 parents - Shots on Target (For)
```

```
#For Shots on Target(For):
```

```
# 0 represents 'Low'
```

```
# 1 represents 'Medium'
```

```
# 2 represents 'High'
```

```

bn.cpt("shots_for")["team_attack": 0, "opponent_defence": 0] = [0.3, 0.4, 0.3]
bn.cpt("shots_for")["team_attack": 0, "opponent_defence": 1] = [0.45, 0.3, 0.25]
bn.cpt("shots_for")["team_attack": 0, "opponent_defence": 2] = [0.65, 0.25, 0.1]
bn.cpt("shots_for")["team_attack": 1, "opponent_defence": 0] = [0.25, 0.3, 0.45]
bn.cpt("shots_for")["team_attack": 1, "opponent_defence": 1] = [0.3, 0.4, 0.3]
bn.cpt("shots_for")["team_attack": 1, "opponent_defence": 2] = [0.5, 0.3, 0.2]

```

```

bn.cpt("shots_for") [{"team_attack": 2, "opponent_defence": 0}] = [0.1, 0.25, 0.65]
bn.cpt("shots_for") [{"team_attack": 2, "opponent_defence": 1}] = [0.2, 0.3, 0.5]
bn.cpt("shots_for") [{"team_attack": 2, "opponent_defence": 2}] = [0.25, 0.5, 0.25]

# CPT for a an intermediate node - Shots on Target(Against)

#For Shots on Target(Against):
# 0 represents 'Low'
# 1 represents 'Medium'
# 2 represents 'High'

bn.cpt("shots_against") [{"team_defence": 0, "opponent_attack": 0}] = [0.3, 0.4, 0.3]
bn.cpt("shots_against") [{"team_defence": 0, "opponent_attack": 1}] = [0.25, 0.3, 0.45]
bn.cpt("shots_against") [{"team_defence": 0, "opponent_attack": 2}] = [0.1, 0.25, 0.65]
bn.cpt("shots_against") [{"team_defence": 1, "opponent_attack": 0}] = [0.45, 0.3, 0.25]
bn.cpt("shots_against") [{"team_defence": 1, "opponent_attack": 1}] = [0.3, 0.4, 0.3]
bn.cpt("shots_against") [{"team_defence": 1, "opponent_attack": 2}] = [0.25, 0.3, 0.45]
bn.cpt("shots_against") [{"team_defence": 2, "opponent_attack": 0}] = [0.65, 0.25, 0.1]
bn.cpt("shots_against") [{"team_defence": 2, "opponent_attack": 1}] = [0.5, 0.3, 0.2]
bn.cpt("shots_against") [{"team_defence": 2, "opponent_attack": 2}] = [0.25, 0.5, 0.25]

# CPT for a Leaf node - Match Outcome

#For Match Outcome:
# 0 represents a 'Loss'
# 1 represents a 'Draw'
# 2 represents a 'Win'

bn.cpt("match_outcome") [{"shots_for": 0, "shots_against": 0, "home_advantage": 1}] = [0.3, 0.4, 0.3]
bn.cpt("match_outcome") [{"shots_for": 0, "shots_against": 0, "home_advantage": 0}] = [0.35, 0.4, 0.25]
bn.cpt("match_outcome") [{"shots_for": 0, "shots_against": 1, "home_advantage": 1}] = [0.4, 0.35, 0.25]
bn.cpt("match_outcome") [{"shots_for": 0, "shots_against": 1, "home_advantage": 0}] = [0.5, 0.3, 0.2]
bn.cpt("match_outcome") [{"shots_for": 0, "shots_against": 2, "home_advantage": 1}] = [0.5, 0.3, 0.2]
bn.cpt("match_outcome") [{"shots_for": 0, "shots_against": 2, "home_advantage": 0}] = [0.6, 0.3, 0.1]
bn.cpt("match_outcome") [{"shots_for": 1, "shots_against": 0, "home_advantage": 1}] = [0.2, 0.3, 0.5]
bn.cpt("match_outcome") [{"shots_for": 1, "shots_against": 0, "home_advantage": 0}] = [0.3, 0.3, 0.4]
bn.cpt("match_outcome") [{"shots_for": 1, "shots_against": 1, "home_advantage": 1}] = [0.25, 0.35, 0.4]
bn.cpt("match_outcome") [{"shots_for": 1, "shots_against": 1, "home_advantage": 0}] = [0.3, 0.4, 0.3]
bn.cpt("match_outcome") [{"shots_for": 1, "shots_against": 2, "home_advantage": 1}] = [0.4, 0.35, 0.25]
bn.cpt("match_outcome") [{"shots_for": 1, "shots_against": 2, "home_advantage": 0}] = [0.5, 0.3, 0.2]

```

```

bn.cpt("match_outcome")[{ "shots_for": 2, "shots_against": 0, "home_advantage": 1}] = [0.1, 0.2, 0.7]
bn.cpt("match_outcome")[{ "shots_for": 2, "shots_against": 0, "home_advantage": 0}] = [0.2, 0.3, 0.5]
bn.cpt("match_outcome")[{ "shots_for": 2, "shots_against": 1, "home_advantage": 1}] = [0.2, 0.3, 0.5]
bn.cpt("match_outcome")[{ "shots_for": 2, "shots_against": 1, "home_advantage": 0}] = [0.3, 0.3, 0.4]
bn.cpt("match_outcome")[{ "shots_for": 2, "shots_against": 2, "home_advantage": 1}] = [0.3, 0.3, 0.4]
bn.cpt("match_outcome")[{ "shots_for": 2, "shots_against": 2, "home_advantage": 0}] = [0.35, 0.35, 0.3]

# CPT for all of the other root nodes

# 0 represents 'Low'
# 1 represents 'Medium'
# 2 represents 'High'

bn.cpt("team_attack")[:] = [0.3, 0.4, 0.3]
bn.cpt("team_defence")[:] = [0.3, 0.4, 0.3]
bn.cpt("opponent_attack")[:] = [0.3, 0.4, 0.3]
bn.cpt("opponent_defence")[:] = [0.3, 0.4, 0.3]

```

In [4]: *#print CPT for home_advantage node*
 bn.cpt("home_advantage")

Out[4]:

home_advantage	
0	1
0.5000	0.5000

In [5]: *#print CPT for shots_for node*
 bn.cpt("shots_for")

Out[5]:

		shots_for		
opponent_defence	team_attack	0	1	2
0	0	0.3000	0.4000	0.3000
	1	0.2500	0.3000	0.4500
	2	0.1000	0.2500	0.6500
1	0	0.4500	0.3000	0.2500
	1	0.3000	0.4000	0.3000
	2	0.2000	0.3000	0.5000
2	0	0.6500	0.2500	0.1000
	1	0.5000	0.3000	0.2000
	2	0.2500	0.5000	0.2500

```
In [6]: #print CPT for shots_against node
bn.cpt("shots_against")
```

Out[6]:

		shots_against		
opponent_attack	team_defence	0	1	2
0	0	0.3000	0.4000	0.3000
	1	0.4500	0.3000	0.2500
	2	0.6500	0.2500	0.1000
1	0	0.2500	0.3000	0.4500
	1	0.3000	0.4000	0.3000
	2	0.5000	0.3000	0.2000
2	0	0.1000	0.2500	0.6500
	1	0.2500	0.3000	0.4500
	2	0.2500	0.5000	0.2500

```
In [7]: #print CPT for match_outcome node
bn.cpt("match_outcome")
```

Out[7]:

			match_outcome		
shots_against	home_advantage	shots_for	0	1	2
0	0	0	0.3500	0.4000	0.2500
		1	0.3000	0.3000	0.4000
		2	0.2000	0.3000	0.5000
	1	0	0.3000	0.4000	0.3000
		1	0.2000	0.3000	0.5000
		2	0.1000	0.2000	0.7000
1	0	0	0.5000	0.3000	0.2000
		1	0.3000	0.4000	0.3000
		2	0.3000	0.3000	0.4000
	1	0	0.4000	0.3500	0.2500
		1	0.2500	0.3500	0.4000
		2	0.2000	0.3000	0.5000
2	0	0	0.6000	0.3000	0.1000
		1	0.5000	0.3000	0.2000
		2	0.3500	0.3500	0.3000
	1	0	0.5000	0.3000	0.2000
		1	0.4000	0.3500	0.2500
		2	0.3000	0.3000	0.4000

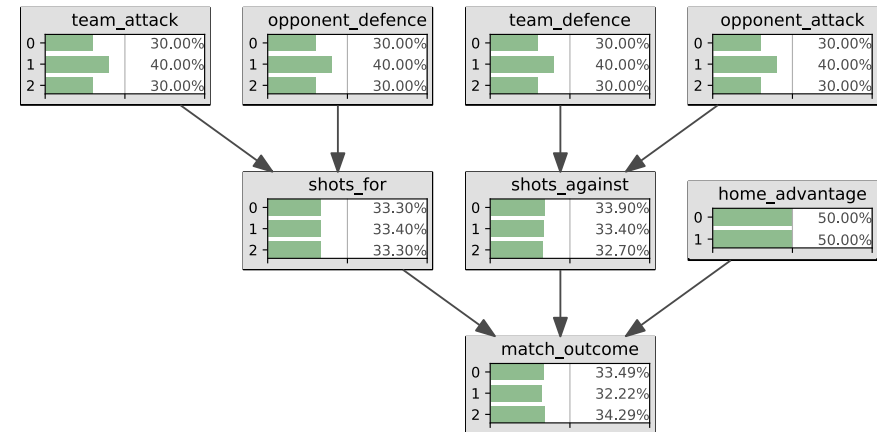
Question 4 - Observational, interventional, and counterfactual reasoning

In [8]: *#Task 4 of Bayesian Coursework - Demonstrating Observational Reasoning*

```
#prior distributions and and inference network
prior = gum.LazyPropagation(bn)
prior.makeInference()
gnb.sideBySide(
    prior.posterior("match_outcome"),
    gnb.getInference(bn , evs = {}),
    captions = [
        "$P(match_outcome)" , "Complete inference with NO evidence"
    ]
)

#posterior distributions and and inference network
# Set evidence of home_advantage to True (1) and shots on target for to High (2)
pos = gum.LazyPropagation(bn)
pos.setEvidence({"home_advantage": 1, "shots_for": 2})
pos.makeInference()
gnb.sideBySide(
    pos.posterior("match_outcome"),
    gnb.getInference(bn , evs = {"home_advantage": 1, "shots_for": 2}),
    captions = [
        "$P(match_outcome|home_advantage = 1,shots_for = 2)" , "Complete inference with evidence = ={home_advantage:1, shots_f"
    ]
)
```

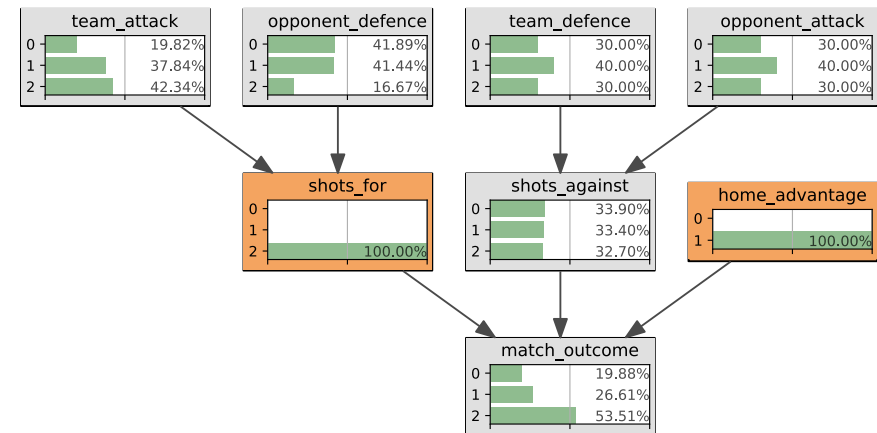

match_outcome		
0	1	2
0.3349	0.3222	0.3429

$$P(\text{match_outcome})$$


Inference in 1.54ms

Complete inference with NO evidence

match_outcome		
0	1	2
0.1988	0.2661	0.5351

$$P(\text{match_outcome} | \text{home_advantage} = 1, \text{shots_for} = 2)$$


Inference in 1.05ms

Complete inference with evidence = {home_advantage:1, shots_for:2}

```
In [9]: #Task 4 of Bayesian Coursework - Demonstrating Interventional Reasoning
model = csl.CausalModel(bn)
cslnb.showCausalImpact(model, "match_outcome", doing="shots_for", values={"shots_for": 2})

#Compare with observational case on same node
```

```
# Set evidence of home_advantage to True (1)
comparison = gum.LazyPropagation(bn)
comparison.setEvidence({"shots_for": 2})
comparison.makeInference()
gmb.sideBySide(
    comparison.posterior("match_outcome"),
    gmb.getInference(bn, evs = {"shots_for": 2}),
    captions = [
        "$P(match_outcome|shots_for = 2)" , "Complete inference with evidence = {shots_for:2}"
    ]
)
```



Causal Model

$$P(\text{match_outcome} \mid \text{do}(\text{shots_for})) = \sum_{\text{home_advantage}, \text{opponent_attack}, \text{shots_against}, \text{team_defence}} P(\text{opponent_attack}) \cdot P(\text{team_defence}) \cdot P(\text{home_advantage}) \cdot P(\text{shots_against} \mid \text{opponent_attack}, \text{team_defence}) \cdot P(\text{match_outcome} \mid \text{home_advantage}, \text{shots_against}, \text{shots_for})$$

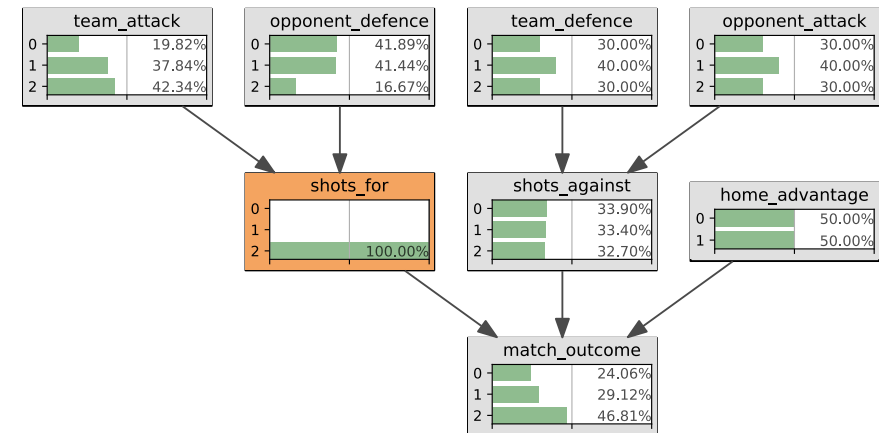
Explanation : Do-calculus computations

match_outcome		
0	1	2
0.2406	0.2912	0.4681

Impact



match_outcome		
0	1	2
0.2406	0.2912	0.4682

$$P(\text{match_outcome} | \text{shots_for} = 2)$$


Inference in 0.81ms

Complete inference with evidence = {shots_for:2}

In [10]: #Task 4 of Bayesian Coursework - Demonstrating Counterfactual Reasoning

#scenario - team has Low shots for and high shots against while playing away. In Hypothetical world , team plays at home.

Counterfactual = gum.BayesNet(bn)

gnb.sideBySide(

gnb.getInference(bn, evs={"home_advantage": 0, "shots_for": 0, "shots_against": 2}),

gnb.getInference(Counterfactual, evs={"home_advantage": 1, "shots_for": 0, "shots_against": 2}),

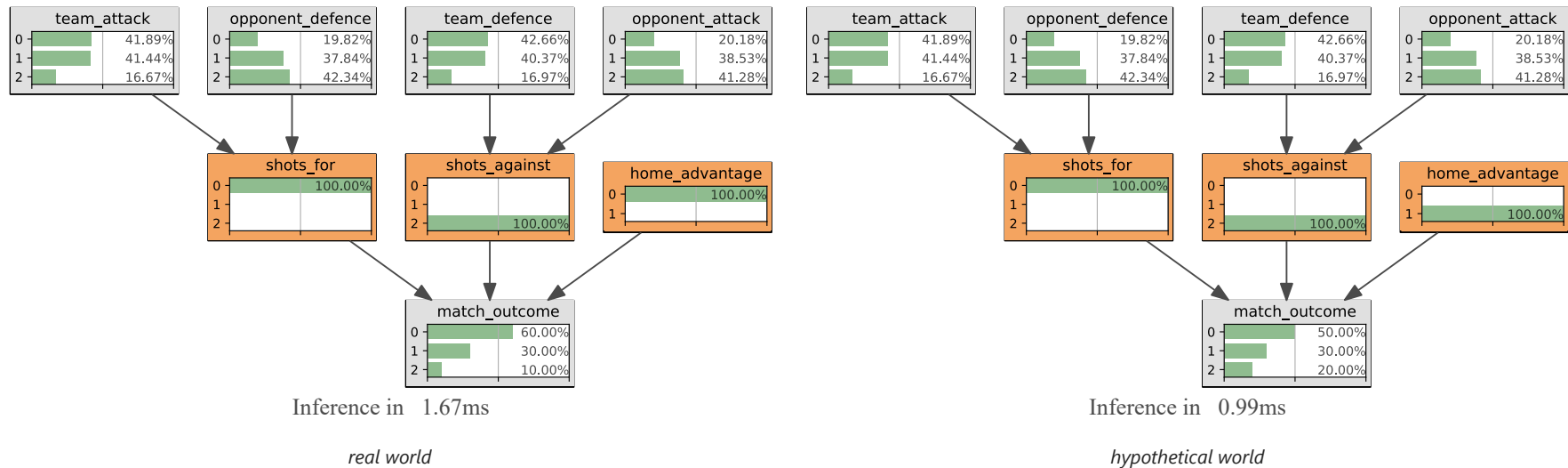
captions=[

"real world",

"hypothetical world"

]

)



Question 5 - From Bayesian Networks to Influence Diagrams

In [11]: #Task 5 of Bayesian Coursework - Adding Decision and Utility nodes for Influence Diagrams

```
#create Influence Diagram
diagram = gum.InfluenceDiagram()

#existing nodes from Bayesian Network - chance nodes
match_outcome = diagram.addChanceNode(gum.LabelizedVariable("match_outcome", "Match Outcome", 3))
shots_for = diagram.addChanceNode(gum.LabelizedVariable("shots_for", "Shots on Target (For)", 3))
shots_against = diagram.addChanceNode(gum.LabelizedVariable("shots_against", "Shots on Target (Against)", 3))
home_advantage = diagram.addChanceNode(gum.LabelizedVariable("home_advantage", "Home Advantage", 2))
team_attack = diagram.addChanceNode(gum.LabelizedVariable("team_attack", "Team Attack Strength", 3))
team_defence = diagram.addChanceNode(gum.LabelizedVariable("team_defence", "Team Defence Strength", 3))
opponent_attack = diagram.addChanceNode(gum.LabelizedVariable("opponent_attack", "Opponent Attack Strength", 3))
opponent_defence = diagram.addChanceNode(gum.LabelizedVariable("opponent_defence", "Opponent Defence Strength", 3))

# Create decision nodes - tactical_plan and substitutions
##For both decision nodes :
# 0 represents 'Defensive'
# 1 represents 'Balanced'
```

```

# 2 represents 'Attacking'

tactical_plan = diagram.addDecisionNode(gum.LabelizedVariable("tactical_plan", "Tactical Plan", 3))
substitutions = diagram.addDecisionNode(gum.LabelizedVariable("substitutions", "Substitutions", 3))

#Create utility nodes - finalResult and net_shots_on_target
finalResult = diagram.addUtilityNode(gum.LabelizedVariable("finalResult", "Final Result", 1))
net_shots_on_target = diagram.addUtilityNode(gum.LabelizedVariable("net_shots_on_target", "Net Shots on Target", 1))

#Add Original arcs from BN - arcs for chance nodes
diagram.addArc(team_attack, shots_for)
diagram.addArc(opponent_defence, shots_for)
diagram.addArc(team_defence, shots_against)
diagram.addArc(opponent_attack, shots_against)
diagram.addArc(shots_for, match_outcome)
diagram.addArc(shots_against, match_outcome)
diagram.addArc(home_advantage, match_outcome)

#Add arcs for decision Nodes
diagram.addArc(tactical_plan, substitutions)
diagram.addArc(tactical_plan, shots_for)
diagram.addArc(tactical_plan, shots_against)
diagram.addArc(substitutions, shots_for)
diagram.addArc(substitutions, shots_against)

#Add arcs for utility Nodes

diagram.addArc(match_outcome, finalResult)
diagram.addArc(shots_for, net_shots_on_target)
diagram.addArc(shots_against, net_shots_on_target)

# Add CPT's for chance nodes - same as nodes in the BN
diagram.cpt("home_advantage").fillWith(bn.cpt("home_advantage"))
diagram.cpt("team_attack").fillWith(bn.cpt("team_attack"))
diagram.cpt("team_defence").fillWith(bn.cpt("team_defence"))
diagram.cpt("opponent_attack").fillWith(bn.cpt("opponent_attack"))
diagram.cpt("opponent_defence").fillWith(bn.cpt("opponent_defence"))
diagram.cpt("match_outcome").fillWith(bn.cpt("match_outcome"))

# Probability changes due to decisions
# 0 represents 'Defensive'

```

```

# 1 represents 'Balanced'
# 2 represents 'Attacking'
tactical_values = {0: -0.045, 1: 0.00, 2: 0.045}
substituiton_impact = {0: -0.025, 1: 0.00, 2: 0.025}

# Function to change probabilities dependeing on decisions
def changeProb(original, change):
    defensive = original[0]
    balanced = original[1]
    attacking = original[2]

    defensive = defensive - change
    attacking = attacking + change

    return [defensive, balanced, attacking]

# Updated CPT for shots_for node based on the decision nodes
for i in range(3):
    for j in range(3):
        original = bn.cpt("shots_for")["team_attack": i, "opponent_defence": j]
        for x in range(3):
            for y in range(3):
                #based on decisions made , calculate total change and change CPT's
                netChange = tactical_values[x] + substituiton_impact[y]
                diagram.cpt("shots_for")["team_attack": i,
                                         "opponent_defence": j, "tactical_plan": x,
                                         "substitutions": y] = changeProb(original, netChange)

# Updated CPT for shots_against node based on the decision nodes
for i in range(3):
    for j in range(3):
        original = bn.cpt("shots_against")["team_defence": i, "opponent_attack": j]
        for x in range(3):
            for y in range(3):
                #based on decisions made , calculate total change and change CPT's
                netChange = tactical_values[x] + substituiton_impact[y]
                diagram.cpt("shots_against")["team_defence": i,
                                             "opponent_attack": j, "tactical_plan": x,
                                             "substitutions": y] = changeProb(original, netChange)

#values for finalResult Utility node

```

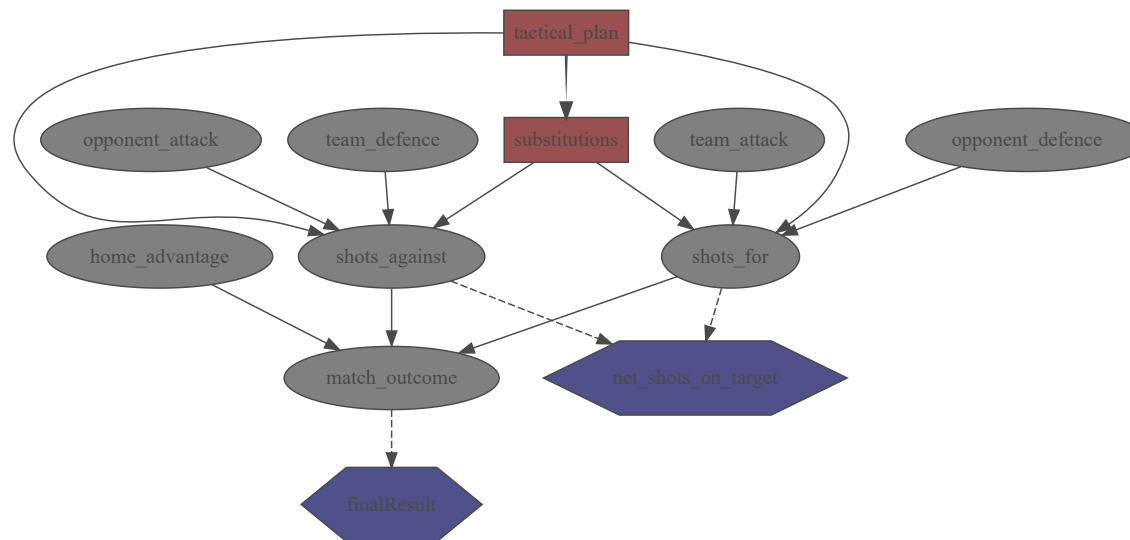
```

# 0 represents a 'Loss;
# 1 represents a 'Draw'
# 3 represents a 'Win
diagram.utility("finalResult")[{ "match_outcome": 0}] = 0
diagram.utility("finalResult")[{ "match_outcome": 1}] = 1
diagram.utility("finalResult")[{ "match_outcome": 2}] = 3

#values for net_shots_on_target Utility node
# determines net shots on target by doing shots for - shots against
diagram.utility("net_shots_on_target")[{ "shots_for": 0, "shots_against": 0}] = 0
diagram.utility("net_shots_on_target")[{ "shots_for": 0, "shots_against": 1}] = -1
diagram.utility("net_shots_on_target")[{ "shots_for": 0, "shots_against": 2}] = -2
diagram.utility("net_shots_on_target")[{ "shots_for": 1, "shots_against": 0}] = 1
diagram.utility("net_shots_on_target")[{ "shots_for": 1, "shots_against": 1}] = 0
diagram.utility("net_shots_on_target")[{ "shots_for": 1, "shots_against": 2}] = -1
diagram.utility("net_shots_on_target")[{ "shots_for": 2, "shots_against": 0}] = 2
diagram.utility("net_shots_on_target")[{ "shots_for": 2, "shots_against": 1}] = 1
diagram.utility("net_shots_on_target")[{ "shots_for": 2, "shots_against": 2}] = 0

gnb.showInfluenceDiagram(diagram)

```



Question 6 - Decision analysis

```
In [13]: #Task 6 of Bayesian Coursework - Testing 2 Plausible decision scenarios

gnb.sideBySide(

#scenario 1 - Team sets up defensively and defensive substitutions are made through the game
    gnb.getInference(diagram, evs={"tactical_plan": 0, "substitutions": 0}),

#scenario 2 - Team sets up offensively and attacking substitutions are made throughout the game
    gnb.getInference(diagram, evs={"tactical_plan": 2, "substitutions": 2}),

    captions=[ "Scenario 1 - Defensive Setup and Substitutions",
                "Scenario 2 - Attacking Setup and Substitutions" ]
)

# Compute optimal decisions and report Maximum Expected Utility

optimal = gum.ShaferShenoyLIMIDInference(diagram)
optimal.makeInference()

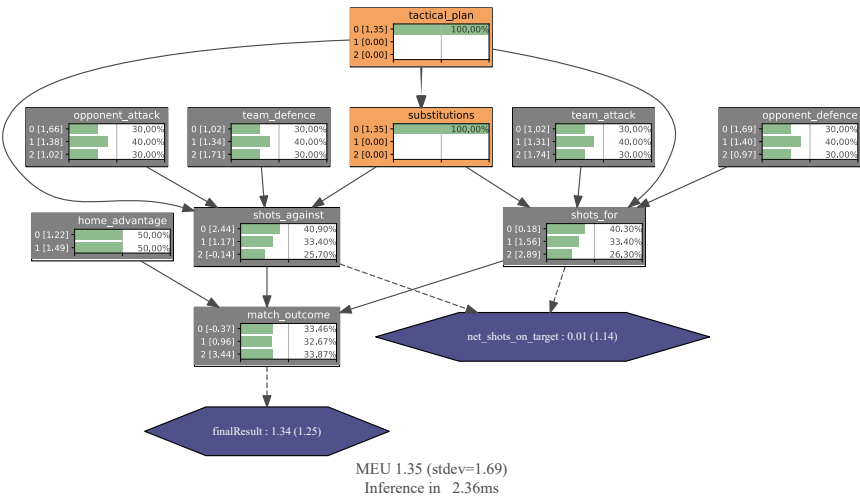
gnb.flow.row(

    #optimal decision for the tactical_plan decision node
    optimal.optimalDecision("tactical_plan"),

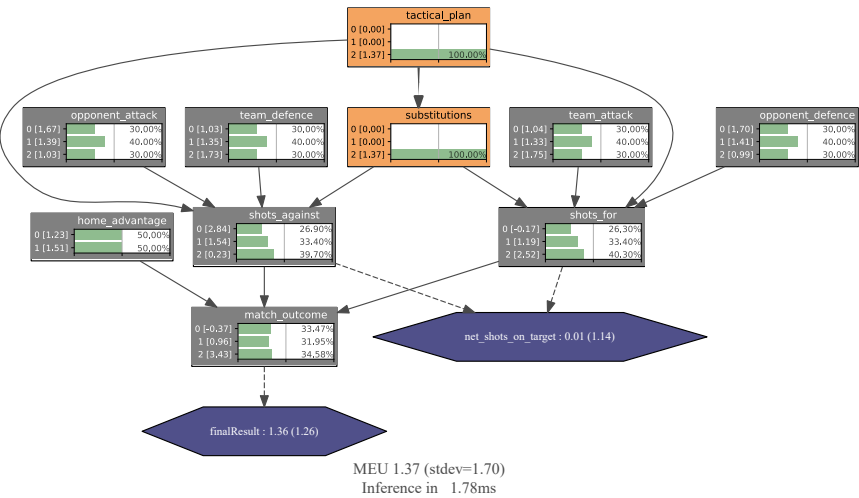
    #optimal decision for the substitutions decision node
    optimal.optimalDecision("substitutions"),

    #Compute the MEU
    f"MEU = {optimal.MEU()['mean']}",

    captions=[
        "Optimal Decision for Tactical Plan",
        "Optimal Decision for Substitutions",
        "Maximum Expected Utility (MEU)"
    ]
)
```

Scenario 1 - Defensive Setup and Substitutions



Scenario 2 - Attacking Setup and Substitutions

tactical_plan		
0	1	2
0.0000	0.0000	1.0000

substitutions		
0	1	2
0.0000	0.0000	1.0000

MEU = 1.3689418000000002

Optimal Decision for Tactical Plan

Optimal Decision for Substitutions

Maximum Expected Utility (MEU)

In []: